```
([01]?\d\d?|2[0-4]\d|25[0-5])\.([01]?\d\d?|2[0-4]\d|25[0-5])\.
([01]?\d\d?|2[0-4]\d|25[0-5])" msg:'Remote File Inclusion
attack'
#To detect inclusions containing PHP function 'include()'
SecRule ARGS "@rx \binclude\s*\([\w|\s]*(ht|f)tps?://"
"msg:'Remote File Inclusion'"
# To detect inclusion ending with '?'
SecRule ARGS "@rx (ft|htt)ps?.*\?+$" msg:'Remote File Inclusion'
```

Miscellaneous security features

You can also block IP addresses by the following command:

```
SecFilterSelective "REMOTE_ADDR" "^192.168.1.1$"
```

If you have an input field *URL* in your comment form, and you want to scan the value of *URL* for the string *c99*, you do it as follows:

```
SecFilterSelective "ARG_url" "c99"
```

```
The following configuration helps fight HTTP fingerprinting, and
accepts only valid protocol versions:
```

```
SecFilterSelective SERVER_PROTOCOL !^HTTP/(0\.9|1\.0|1\.1)$
```

The following configuration allows supported request methods only, and helps fight XST attacks:

```
SecFilterSelective REQUEST_METHOD !^(GET|HEAD|POST)$
```

Often during the reconnaissance phase, attackers look for the Web server identity and version. Web servers typically send their identity with every HTTP response, in the *Server* header. Apache is particularly helpful here; it not only sends its name and full version, by default, but also allows server modules to append their versions. Here, you can confuse the attackers by using something like: *SecServerSignature "Microsoft-IIS/5.0"*

PHP code cannot be injected directly, but it may be possible to have code recorded on disk to be executed later, using an LFI attack. The following rule will detect such an injection attempt, but will ignore XML documents, which use similar syntax:

```
SecRule ARGS "@rx <\?(?!xml)"
```

Logging

There are three places where, depending on the configuration, you may find *mod_security* logging information:

- *Mod_security* debug log: If enabled via the *SecFilterDebugLevel* and *SecFilterDebugLog* directives, it contains a large number of entries for every request processed. Each log entry is associated with a log level, which is a number from 0 (no messages at all) to 4 (maximum logging). You normally keep the debug log level at 0, and increase it only when you are debugging your rule set.
- Apache error log: Some of the messages from the debug log will make it into the Apache error log (even if you set *mod_security* debug log level to 0). These are the messages that require an administrator's attention, such as information about requests being rejected.
- *Mod_security* audit log: When audit logging is enabled (using the *SecAuditEngine* and *SecAuditLog* directives), *mod_security* can record each request (and its body, provided request body buffering is enabled) and the corresponding response headers.

Here is an example of an error message resulting from invalid content discovered in a cookie:

```
[Tue Jun 26 17:44:36 2011] [error] [client 127.0.0.1]
mod_security: Access denied with code 500. Pattern match
"!(^$|^[a-zA-Z0-9]+$)"
at COOKIES_VALUES(sessionid) [hostname "127.0.0.1"]
[uri "/test.php"] [unique_id 3434fvnij54jktynv45fC8QQQQAB]
```

The message indicates that the request was rejected (*"Access denied"*) with an HTTP 500 response because the content of the cookie sessionid contained content that matched the pattern *!(^$|^[a-zA-Z0-9]+$)*. (The pattern allows a cookie to be empty, but if it is not, it must consist only of one or more letters and digits.)

Note: I once again stress that neither LFY nor myself are responsible for the misuse of the information given here. Any attack techniques described here are meant to give you the knowledge that you need to protect your own infrastructure. Please use the tools and techniques sensibly.

This article has just scratched the surface of *mod_security*. For more details on rule writing and other important directives, please refer to the resource in the 'Further reading' section.

We will deal with other ways to secure Apache in the next article. Always remember: Know hacking, but no hacking. END

Further reading

'ModSecurity Handbook' by Ivan Ristic. This is a must-read book for anyone interested in this topic.

By: Arpit Bajpai

The author is a FOSS enthusiast and loves to troubleshoot in the information security domain, while exploring new exploits and system vulnerabilities. You can send your queries and constructive feedback on the article to *abajpai75 AT yahoo DOT com*.